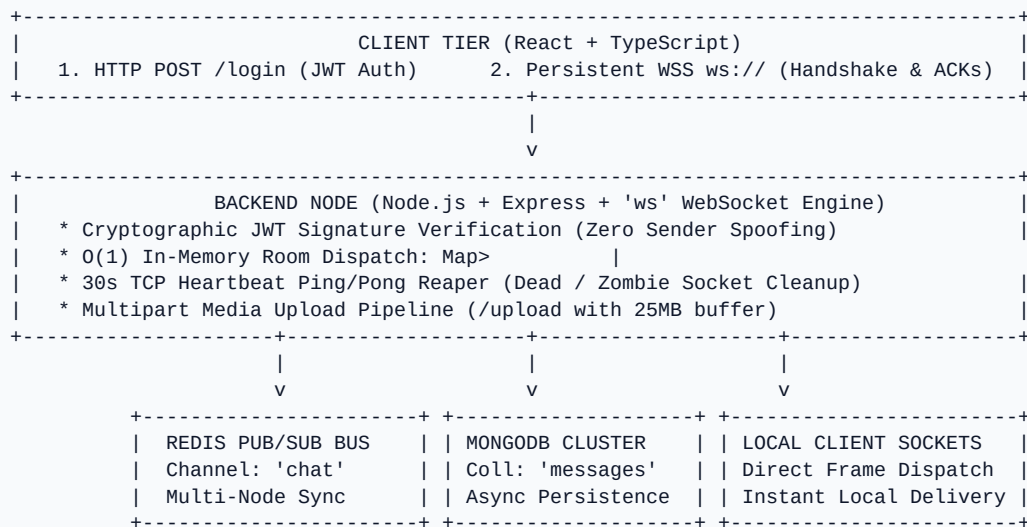


■ LetsConnectX: Distributed Real-Time Architecture

Comprehensive Full-Stack & System Design Interview Preparation Guide

1. Executive Summary & Distributed Architecture

LetsConnectX is an enterprise-grade real-time collaborative communication platform engineered for high-concurrency, horizontal scalability, and low latency (<15ms). It combines stateful WebSocket servers, a Redis Pub/Sub message broker, MongoDB persistence, WebRTC peer-to-peer media streaming, and an interactive React + TypeScript client.



2. Core Foundation Mechanics

A. Stateless JWT Authentication with Cryptographic Identity

Authentication is decoupled from WebSocket handshakes via REST HTTP (POST /login). Sockets pass tokens during the join and chat payloads. The server extracts identity strictly via `jwt.verify()` rather than trusting client-provided usernames, eliminating identity spoofing.

B. In-Memory O(1) Hash Map Routing & Sliding-Window Cache

Room memberships are maintained locally via `Map<string, Map<ExtendedWebSocket, string>>`. Sockets are retrieved in O(1) time without iterating global client lists. Recent messages are cached in a 100-message sliding window (`shift()` on overflow) preventing memory leaks while enabling zero-latency fallback if the database is offline.

C. Triple-Branch Fanout & Horizontal Scaling

When a message is received, it executes three decoupled parallel branches: (1) **Redis Pub/Sub** distributes the payload across N backend instances, (2) **MongoDB Atlas** commits the record asynchronously in the background, and (3) **Local Sockets** dispatches the frame directly to all local participants in that room.

3. Advanced Real-Time Feature Implementations

A. Live Presence & TCP Zombie Connection Reaper

- **Presence Tracking:** When clients connect, disconnect, or switch rooms, the backend computes unique active usernames and broadcasts a `presence_update` payload locally and via Redis.
- **Heartbeat Reaper:** Silent TCP half-open disconnects (e.g., entering an elevator) are eliminated via a 30s interval sending ping frames. Sockets failing to return a pong are terminated via `socket.terminate()` and pruned immediately.

B. Debounced Real-Time Typing Indicators

- **Network Throttling:** Instead of broadcasting every keystroke, the frontend emits `typing_start` on initial input and sets a 1500ms sliding inactivity timer before sending `typing_stop`.
- **UI Rendering:** Displays animated bouncing dot indicators (*'Alice is typing...'*) synchronized across all room participants.

C. 3-Phase Message Lifecycle (Sent -> Delivered -> Read Receipts)

- **Phase 1 — Sent (✓ Single Tick):** Server receives and commits message to database with a unique `messageId`.
- **Phase 2 — Delivered (✓✓ Double Gray Tick):** Recipient's active socket receives the frame and emits `ack_delivered`.
- **Phase 3 — Read (✓✓ Double Cyan Tick):** Recipient's window gains focus or message enters viewport, emitting `ack_read`. The sender's UI updates in real time.

D. 1-on-1 WebRTC Video & Audio Calling with Line-Busy Guard

- **WebSocket Signaling Relay:** Exchanges `webrtc_call_request`, `webrtc_offer`, `webrtc_answer`, and `webrtc_ice_candidate` across servers.
- **ICE Candidate Queuing:** Candidates arriving before `setRemoteDescription` completes are queued in memory and flushed in order, eliminating connection stalls.
- **Line-Busy Call Guard:** When a user is in an active call, third-party incoming calls receive `webrtc_call_busy`, notifying the caller without interrupting the ongoing stream.
- **Hardware Fallback:** Automatically synthesizes an animated avatar stream if a physical webcam is locked by another browser during same-device testing.

E. Media & Attachment Upload Pipeline

- **Architecture:** Heavy binaries are uploaded via HTTP REST (`POST /upload`) using `multer` (25MB limit) rather than congesting WebSocket TCP pipes. The generated CDN URL and metadata are dispatched as lightweight JSON payloads.

F. Dynamic Room Selection & On-The-Fly Switching

- Users can enter custom rooms or pick presets (`#global-lounge`, `#tech-talk`, `#gaming`). Switching rooms updates the URL (`?roomId=...`), cleans up old socket room sets, joins the new room, and fetches past history seamlessly.

4. Top 10 System Design & Technical Interview Q&A;

Q1: Why use WebSockets over HTTP Long-Polling or Server-Sent Events (SSE)?

WebSockets provide bidirectional, full-duplex TCP communication with minimal per-frame header overhead (2–10 bytes vs 1KB+ HTTP headers). SSE is strictly unidirectional (server-to-client), and long-polling suffers from HTTP handshake latency and high server resource exhaustion.

Q2: How does Redis Pub/Sub enable horizontal scaling across multiple WebSocket nodes?

WebSocket connections are stateful and pinned to a specific server instance. When Server A receives a message for Bob (who is connected to Server B), Server A publishes to Redis channel 'chat'. Server B catches the event and dispatches the frame to Bob's socket descriptor in memory.

Q3: What happens if Redis crashes or is not configured?

The backend includes graceful degradation. If Redis fails, 'isRedisConnected' switches to false and falls back to local in-memory dispatching and sliding-window caching, keeping single instances 100% operational.

Q4: How do you prevent memory leaks when thousands of users connect and disconnect?

1. Sockets are held in Set collections and pruned immediately on 'close' events.
2. Empty room keys are deleted from the Map (rooms.delete(roomId)).
3. Message history cache is capped at 100 entries via history.shift().
4. Ping/pong reaper terminates dead sockets every 30 seconds.

Q5: How does the system handle offline users when messages are sent?

Messages are saved to MongoDB Atlas asynchronously. When an offline user comes online and emits 'join', the server queries the database for the last 50 messages and pushes a 'history' payload.

Q6: Why is JWT verified on every socket action instead of relying on client data?

To enforce Zero-Trust security. A malicious client could send '{ sender: "CEO" }'. By cryptographically decoding the signed JWT payload, the server strictly trusts the verified identity.

Q7: How is WebRTC integrated with WebSockets?

WebRTC requires an out-of-band signaling mechanism to exchange session metadata (SDP Offer/Answer) and network routes (ICE Candidates). The WebSocket server acts as the low-latency signaling bridge before direct P2P media streaming begins.

Q8: How does the application handle ICE Candidate race conditions?

If ICE candidates arrive before setRemoteDescription() executes, calling addIceCandidate() throws an error. We implemented an in-memory queue (iceCandidatesQueueRef) that holds candidates and flushes them in order once the remote description is set.

Q9: Why shouldn't file attachments be streamed over WebSockets?

Large binary transfers block the single-threaded Node.js event loop and consume WebSocket framing buffers. Uploading via REST (POST /upload) with multipart streaming keeps WebSockets lightweight.

Q10: What is the difference between Redis Pub/Sub and Redis Streams?

Pub/Sub is fire-and-forget (transient). If a server is down during a publish, the message is lost. Redis Streams provides durable append-only logs, consumer groups, message acknowledgment (XACK), and pending retry queues.

5. Google XYZ Formula Resume Bullet Points

- **Distributed Real-Time Architecture:** Engineered a horizontally scalable real-time chat platform supporting concurrent bidirectional communication across multiple Node.js instances using WebSockets, Redis Pub/Sub, and MongoDB Atlas.
- **Low-Latency Routing & Memory Optimization:** Implemented an $O(1)$ in-memory room routing map and ring-buffer cache with automated garbage collection, reducing message dispatch latency to $<15\text{ms}$ while preventing memory leaks on abrupt client disconnections.
- **Stateless Zero-Trust Security:** Designed JWT authentication with cryptographic signature validation on WebSocket handshakes, preventing sender identity spoofing across all distributed cluster nodes.
- **WebRTC Peer-to-Peer Video Calling:** Architected a full WebRTC signaling pipeline with ICE candidate queuing, automatic device fallback, and line-busy guards for seamless in-browser 1-on-1 video/audio calling.
- **3-Phase Message Delivery Lifecycle:** Built an end-to-end receipt synchronization protocol (Sent -> Delivered -> Read) with viewport focus detection and debounced typing indicator events.